Laravel + MySQL Technical Assessment

Task: Multi-Tenant Inventory Management System with Real-Time Stock Reconciliation

Background:
You are required to build a multi-tenant inventory management system in Laravel backed by MySQL. Each tenant (company) has its own isolated data, but all tenants share the same database schema. The system must support high-concurrency stock movements and produce accurate stock levels at all times.

Requirements:

1. **Multi-Tenancy**
    - Implement tenant isolation using a `tenant_id` column on all relevant tables (do not use separate databases).
    - Ensure a global scope automatically filters queries by the current tenant.
    - Provide a way to switch tenants at runtime (e.g., via middleware resolving from subdomain or header).
2. **Schema**
   Design and migrate the following tables:
    - `tenants` (id, name, subdomain, created_at)
    - `products` (id, tenant_id, sku, name, unit_price, created_at)
    - `warehouses` (id, tenant_id, name, location, created_at)
    - `stock_movements` (id, tenant_id, product_id, warehouse_id, type ENUM('in','out','transfer'), quantity, reference, created_at)
    - `stock_levels` (id, tenant_id, product_id, warehouse_id, quantity, updated_at)
    - Add a unique constraint on `(tenant_id, sku)` for products and `(tenant_id, product_id, warehouse_id)` for stock_levels.
3. **Business Logic**
    - When a stock movement is recorded, update `stock_levels` accordingly inside a database transaction with row-level locking (`SELECT ... FOR UPDATE`).
    - Prevent negative stock: if an `out` or `transfer` movement would result in negative quantity, throw a domain exception and roll back.
    - A `transfer` movement must atomically decrement the source warehouse and increment the destination warehouse. Use a `meta` JSON column on `stock_movements` to store `from_warehouse_id` and `to_warehouse_id` for transfers.
4. **Concurrency**

- Write a test that fires 100 concurrent `out` movements of quantity 1 against a product with initial stock of 50 in the same warehouse.
- Assert that exactly 50 succeed and 50 fail, and that final stock is 0.
- Use Laravel's queue or parallel testing tools to simulate concurrency.

5. **Reporting**

   - Build an artisan command `inventory:reconcile {tenant}` that:
     - Recomputes `stock_levels` from `stock_movements` for the given tenant.
     - Compares recomputed values against current `stock_levels`.
     - Logs any discrepancies with product, warehouse, expected, and actual quantities.
   - The command must be idempotent and safe to run in production.

6. **API**

   - Provide REST endpoints (with Form Request validation and API Resources):
     - `POST /api/stock-movements` — record a movement.
     - `GET /api/stock-levels` — list stock levels, filterable by `product_id` and `warehouse_id`, paginated.
     - `GET /api/products/{sku}/history` — paginated movement history for a SKU within the current tenant.

7. **Performance**

   - Add appropriate indexes to support the queries above.
   - The `GET /api/stock-levels` endpoint must not exceed 200ms with 1,000,000 stock_level rows for a single tenant. Explain your indexing strategy in a short README.

8. **Deliverables**

   - A GitHub repository (or zip) with the Laravel project.
   - Migration files, models, services, controllers, form requests, resources, tests, and the artisan command.
   - A `README.md` explaining:
     - Setup steps.
     - Architectural decisions (especially around concurrency and multi-tenancy).
     - Indexing strategy.
     - How to run the concurrency test.

Evaluation Criteria:

- Correctness of stock reconciliation under concurrency.
- Clean separation of concerns (service layer, not fat controllers).
- Test coverage, especially the concurrency test.

- Query performance and index design.
- Code readability and adherence to Laravel conventions.